
Mục lục

1	Tiền xử lý và trực quan hóa dữ liệu	5
1.1	Nạp và tiền xử lý dữ liệu thực nghiệm	5
1.2	Trực quan hóa phân bố dữ liệu thực nghiệm	7
2	Lan truyền sai số giải tích và tự động hóa	11
2.1	Lan truyền sai số giải tích thông qua đại số máy tính	11
2.2	Tự động hóa lan truyền sai số với thư viện uncertainties	13
3	Hồi quy động học và ma trận hiệp phương sai	15
3.1	Thuật toán hồi quy và trích xuất ma trận hiệp phương sai	15
3.2	Hồi quy tuyến tính và đánh giá chỉ số R-bình phương	17
3.3	Trực quan hóa mô hình hồi quy động học	19
4	Kỹ thuật chẩn đoán mô hình nâng cao	23
4.1	Đánh giá mô hình với tiêu chuẩn Chi bình phương rút gọn	23
4.2	Phân tích phần dư và kỹ thuật làm trơn LOWESS	25
4.3	Kiểm chứng độ bất định bằng phương pháp mô phỏng Monte Carlo	27
A	Cẩm nang tra cứu nhanh thuật toán (Methodological Quick Reference)	31
	Tài liệu tham khảo	35

Danh sách hình vẽ

1.1	Trực quan hóa phân bố dữ liệu thực nghiệm ban đầu của độ dịch chuyển và vận tốc dẫn xuất.	9
3.1	Trực quan hóa đồ thị hồi quy kết hợp hợp thông tin tham số.	21
4.1	Phân tích phần dư và chẩn đoán xu hướng hệ thống thông qua kỹ thuật làm trơn LOWESS.	26
4.2	Phân bố của giá trị gia tốc trọng trường thu được thông qua mô phỏng Monte Carlo.	29

Tiền xử lý và trực quan hóa dữ liệu

1.1 Nạp và tiền xử lý dữ liệu thực nghiệm

Việc ứng dụng ngôn ngữ lập trình Python mang lại hiệu quả cao trong khâu tự động hóa quá trình nạp dữ liệu (data ingestion) từ các thiết bị đo lường. Trong thực nghiệm vật lý, các hệ thống cảm biến tự động thường trích xuất dữ liệu dưới dạng bảng với mật độ lấy mẫu thời gian cao. Việc thiết lập các scripts không chỉ hỗ trợ xử lý dữ liệu hàng loạt nhằm duy trì tính nhất quán, mà còn giúp giảm thiểu đáng kể rủi ro sai lệch phương pháp thường phát sinh khi thao tác thủ công trên các phần mềm bảng tính truyền thống.

Cơ sở vật lý 1.1: Đặc tính động học của vật thể rơi tự do

Bài toán rơi tự do được xem xét như một ví dụ minh họa (illustrative case study) cho phương pháp xử lý dữ liệu đo lường tổng quát. Đặc trưng cơ bản của hệ thống này là gia tốc chuyển động duy trì giá trị không đổi ($a = g$). Theo đó, phương trình độ dịch chuyển $s(t)$ tuân theo quy luật đa thức bậc hai đối với thời gian, trong khi đại lượng vận tốc tức thời $v(t)$ biến thiên tuyến tính. Việc tự động hóa khâu nạp và tiền xử lý mảng dữ liệu tọa độ từ cảm biến là nền tảng cốt lõi để kiểm chứng các đặc tính vật lý này.

Sai lầm thường gặp 1.1: Hiện tượng sai lệch cấu trúc do định dạng khu vực (local formatting)

Dữ liệu thô xuất ra từ các cảm biến lấy mẫu tự động (như hệ thống CASSY) thường chịu ảnh hưởng của các tiêu chuẩn định dạng khu vực (local formatting) – điển hình như việc sử dụng dấu phẩy thay cho dấu chấm ở phần thập phân, hoặc dùng dấu chấm phẩy làm ký tự phân cách (delimiter). Việc bỏ sót bước chuẩn hóa định dạng trong quá trình nạp tệp dễ dẫn đến hiện tượng sai lệch cấu trúc (structural data drift), khiến hệ thống phân tích nhận diện nhầm mảng giá trị số học (float) thành chuỗi ký tự (string). Nhằm duy trì tính tái lập (reproducibility) và khắc phục rủi ro này, ta tiến hành nạp dữ liệu trực tiếp từ các tệp lưu trữ ngoại vi (chẳng hạn định dạng `.csv`), kết hợp khai báo tường minh các tham số phân tách thông qua các hàm nạp dữ liệu chuẩn.

Thư viện pandas cung cấp cấu trúc dữ liệu DataFrame mạnh mẽ, cho phép nạp trực tiếp luồng dữ liệu thực

nghiệm từ tệp lưu trữ và chuyển đổi liền mạch sang định dạng mảng (array) của thư viện numpy để phục vụ các phép toán vector hóa ở những bước tiếp theo.

Python Implementation: Automated data ingestion and array conversion

```
1 import pandas as pd
2 import numpy as np
3
4 # Ingesting the master dataset directly from an external CSV file
5 df = pd.read_csv('kinematics_data.csv')
6 time_data = df['Time_s'].values
7 displacement_data = df['Displacement_m'].values
```

Sau khi dữ liệu cơ sở được chuyển đổi về định dạng mảng, quy trình phân tích sẽ tiến hành tính toán các đại lượng động học thứ cấp. Đối với các hệ thống tự động, biến độc lập (thời gian) và biến phụ thuộc (độ dịch chuyển) thường được ghi nhận trực tiếp. Do đó, vận tốc tức thời cần được ước lượng gián tiếp thông qua các kỹ thuật toán học.

Khai triển toán học 1.1: Phương pháp vi phân số học cho vận tốc tức thời

Giả sử tập dữ liệu thực nghiệm được biểu diễn dưới dạng các cặp giá trị rời rạc (t_i, s_i) . Theo định nghĩa của đạo hàm, vận tốc tức thời v trên một khoảng thời gian giữa hai điểm đo liên tiếp có thể được ước lượng thông qua phương pháp vi phân số học (numerical differentiation):

$$v_i \approx \frac{\Delta s_i}{\Delta t_i} = \frac{s_{i+1} - s_i}{t_{i+1} - t_i} \quad (1.1)$$

Giá trị xấp xỉ này phản ánh sát nhất vận tốc tại thời điểm nằm chính giữa khoảng lấy mẫu. Vì vậy, ta cần thiết lập một hệ trục thời gian trung gian (mid-point times) tương ứng với mảng vận tốc vừa được dẫn xuất:

$$t_{\text{mid},i} = t_i + \frac{\Delta t_i}{2} \quad (1.2)$$

Sự biến thiên Δs và Δt trên toàn bộ tập dữ liệu được tự động ước lượng thông qua hàm `np.diff`. Việc ứng dụng các hàm tính toán mảng (vectorized operations) trên môi trường Python giúp hạn chế sự phụ thuộc vào các vòng lặp thủ công, từ đó tối ưu hóa tốc độ xử lý đối với những tập dữ liệu đo lường có quy mô lớn.

Python Implementation: Numerical differentiation for kinematic derivation

```
1 # Perform numerical differentiation to approximate instantaneous velocity: v =
   ↪ (s_{i+1} - s_i) / (t_{i+1} - t_i)
2 delta_s = np.diff(displacement_data)
3 delta_t = np.diff(time_data)
4 velocity_data = delta_s / delta_t
5
```

```

6 # Compute corresponding mid-point times for the derived velocity array
7 time_midpoints = time_data[:-1] + delta_t / 2.0

```

1.2 Trực quan hóa phân bố dữ liệu thực nghiệm

Quá trình trực quan hóa dữ liệu là công đoạn cơ bản trong quy trình phân tích của các hệ thống đo lường tự động. Thay vì làm việc với các bảng dữ liệu quy mô nhỏ, các mảng dữ liệu được nạp từ tệp .csv (như thời gian, độ dịch chuyển và vận tốc) cần được ánh xạ vào không gian tọa độ. Thao tác này hỗ trợ việc đánh giá khách quan đặc tính quỹ đạo và mức độ phân tán của dữ liệu trước khi tiến hành tính toán lan truyền sai số và hồi quy mô hình. Đối với bài toán minh họa rơi tự do, bước này giúp xác nhận sơ bộ tính chất parabol của tọa độ và tính tuyến tính của đại lượng vận tốc.

Cơ sở vật lý 1.2: Khái niệm điểm dữ liệu và vùng bất định đo lường

Trong khuôn khổ của vật lý thực nghiệm và đo lường học, một điểm dữ liệu (data point) không được định nghĩa là một tọa độ vô hướng đơn thuần trên mặt phẳng toán học. Bản chất của mỗi điểm dữ liệu là một vùng bất định (uncertainty region), được giới hạn bởi các giá trị sai số hệ thống (systematic errors) từ thiết bị cảm biến. Đối với các hệ thống lấy mẫu tự động, giới hạn độ phân giải không gian và thời gian tạo thành các ranh giới vi mô bao quanh mỗi kết quả ghi nhận.

Để triển khai nguyên lý này trên môi trường lập trình, thư viện matplotlib được sử dụng nhằm mục đích cấu trúc hóa các đại lượng động học thành đối tượng đồ họa. Việc cấu hình các tham số hiển thị toàn cục (như phông chữ chuẩn LaTeX và độ phân giải) là yêu cầu thiết yếu nhằm đảm bảo đồ thị đầu ra đáp ứng tiêu chuẩn xuất bản khoa học.

Python Implementation: High-resolution visualization of sensor data

```

1 import matplotlib.pyplot as plt
2
3 # Global configuration for high-resolution scientific plotting
4 plt.rcParams.update({
5     "text.usetex": True,
6     "font.family": "serif",
7     "font.serif": ["Computer Modern Roman"],
8     "figure.dpi": 150,
9     "axes.labelsize": 12,
10    "font.size": 12,
11    "legend.fontsize": 10,
12    "xtick.labelsize": 10,
13    "ytick.labelsize": 10,

```

```

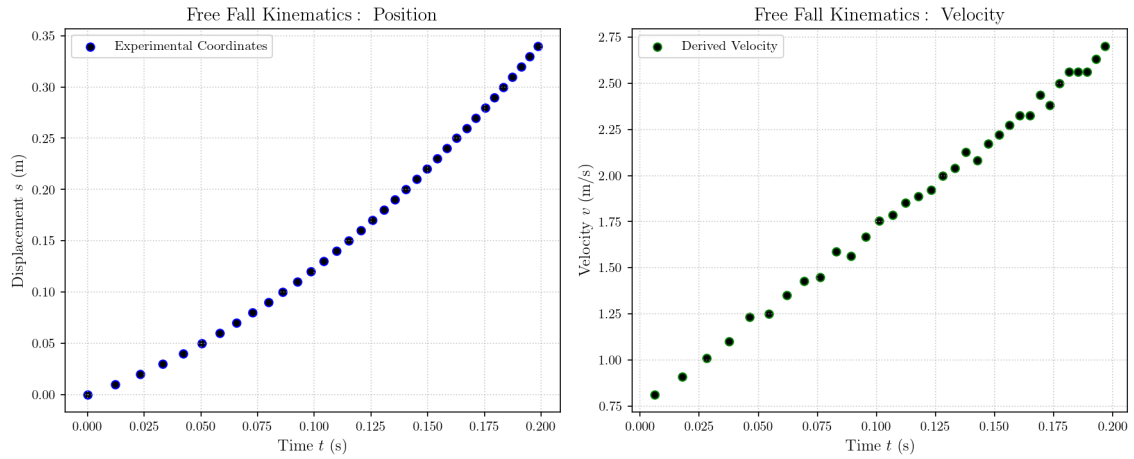
14     "axes.titlesize": 14,
15     "lines.linewidth": 1.5,
16     "grid.alpha": 0.5,
17     "grid.linestyle": "--"
18 })
19
20 # Setup graphical structures
21 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
22
23 # Visualization 1: Displacement vs Time
24 ax1.scatter(time_data, displacement_data, color='black', edgecolor='blue',
25             label=r'$\mathrm{Experimental \ Coordinates}$')
26 ax1.set_xlabel(r'$\mathrm{Time} \ t \ (\mathrm{s})$')
27 ax1.set_ylabel(r'$\mathrm{Displacement} \ s \ (\mathrm{m})$')
28 ax1.set_title(r'$\mathrm{Free \ Fall \ Kinematics: \ Position}$')
29 ax1.legend()
30 ax1.grid(True, linestyle=':', alpha=0.7)
31
32 # Visualization 2: Velocity vs Time
33 ax2.scatter(time_midpoints, velocity_data, color='black', edgecolor='green',
34             label=r'$\mathrm{Derived \ Velocity}$')
35 ax2.set_xlabel(r'$\mathrm{Time} \ t \ (\mathrm{s})$')
36 ax2.set_ylabel(r'$\mathrm{Velocity} \ v \ (\mathrm{m/s})$')
37 ax2.set_title(r'$\mathrm{Free \ Fall \ Kinematics: \ Velocity}$')
38 ax2.legend()
39 ax2.grid(True, linestyle=':', alpha=0.7)
40
41 plt.tight_layout()
42 plt.show()

```

Đoạn mã trên minh họa kỹ thuật thiết lập biểu đồ song song (subplots) cho cả hai nhóm đại lượng: vị trí đo lường trực tiếp và vận tốc dẫn xuất. Việc định dạng nhãn trục tọa độ được thực hiện thông qua cấu trúc chuỗi thô (raw strings) mang tiền tố r kết hợp cú pháp toán học LaTeX. Ở giai đoạn này, không gian phân bố vĩ mô của tập dữ liệu đã được xác lập, tạo nền tảng trực quan để lồng ghép các đường cong mô hình lý thuyết và thanh sai số cục bộ trong các bước phân tích chuyên sâu.

Sai lầm thường gặp 1.2: Hiện tượng che lấp thanh sai số trong không gian dữ liệu mật độ cao

Bước thiết lập đồ thị đối với các mảng dữ liệu lấy mẫu tự động thường tiềm ẩn rủi ro về mặt biểu diễn thị giác. Do số lượng điểm quan trắc lớn và độ bất định thiết bị ở mức vi mô, việc rải thanh sai số trực tiếp lên toàn bộ đồ thị vĩ mô dễ gây ra hiện tượng nhiễu hình ảnh, hoặc các thanh sai số bị che lấp bởi chính các điểm đánh dấu (marker). Phương pháp biểu diễn này làm giảm khả năng nhận



Hình 1.1: Trực quan hóa phân bố dữ liệu thực nghiệm ban đầu của độ dịch chuyển và vận tốc dẫn xuất.

diện các giới hạn sai số đo lường. Để khắc phục, một giải pháp đồ họa tối ưu là duy trì việc vẽ điểm ở góc nhìn toàn cảnh, đồng thời sử dụng kỹ thuật trục phụ nội tuyến (inset axes) để phóng to một vùng dữ liệu cục bộ (phương pháp này sẽ được tích hợp chi tiết tại chương hồi quy mô hình), giúp minh chứng rõ ràng sự hiện diện của giới hạn phân giải thiết bị.

Lan truyền sai số giải tích và tự động hóa

2.1 Lan truyền sai số giải tích thông qua đại số máy tính

Trong quy trình đo lường học tính toán, nhiều đại lượng vật lý cốt lõi không được ghi nhận trực tiếp từ thiết bị cảm biến mà được ước lượng gián tiếp thông qua các hàm toán học đa biến. Phương pháp vi phân toàn phần (total differential method) là công cụ giải tích nền tảng được ứng dụng nhằm đánh giá độ bất định (uncertainty) của các phép đo gián tiếp dựa trên nguyên lý lan truyền sai số (error propagation).

Khi áp dụng phương pháp luận tổng quát này vào bài toán minh họa động học rơi tự do, vận tốc tức thời v được dẫn xuất từ sự biến thiên của không gian tọa độ s và chuỗi thời gian t tại hai điểm đo kế tiếp (điểm i và điểm j , với $j = i + 1$). Biểu thức toán học cốt lõi được định nghĩa như sau:

$$v = \frac{s_j - s_i}{t_j - t_i} \quad (2.1)$$

Mức độ lan truyền độ bất định từ các biến số đầu vào (bản thân chúng đã chứa đựng sai số hệ thống từ thiết bị đo) sang đại lượng dẫn xuất v được đánh giá chặt chẽ thông qua việc khai triển vi phân toàn phần.

Khai triển toán học 2.1: Khai triển vi phân toàn phần cho đại lượng dẫn xuất

Theo nguyên lý lan truyền độ bất định chuẩn (standard error propagation), độ bất định tổng hợp u_v của đại lượng dẫn xuất được định nghĩa thông qua căn bậc hai của tổng bình phương các vi phân riêng phần (partial differentials). Từ phương trình 2.1, ta có thể thiết lập biểu thức giải tích tổng quát:

$$u_v = \sqrt{\left(\frac{\partial v}{\partial s_i} u_{s_i}\right)^2 + \left(\frac{\partial v}{\partial s_j} u_{s_j}\right)^2 + \left(\frac{\partial v}{\partial t_i} u_{t_i}\right)^2 + \left(\frac{\partial v}{\partial t_j} u_{t_j}\right)^2} \quad (2.2)$$

Trong đó, các hệ số nhạy (sensitivity coefficients) biểu thị mức độ tác động của từng biến độc lập lên kết quả ước lượng cuối cùng, tương ứng với các đạo hàm riêng (partial derivatives) được khai

triển. Đối với các biến số tại thời điểm kết thúc (s_j và t_j), hệ thống đạo hàm này được xác định cụ thể:

$$\frac{\partial v}{\partial s_j} = \frac{1}{t_j - t_i} \quad (2.3)$$

$$\frac{\partial v}{\partial t_j} = -\frac{s_j - s_i}{(t_j - t_i)^2} \quad (2.4)$$

Việc thiết lập các biểu thức giải tích cung cấp một cơ sở toán học tường minh về mức độ đóng góp của từng đại lượng thành phần vào độ bất định tổng quát. Mặc dù vậy, quy trình thực thi này bộc lộ những giới hạn nhất định khi mở rộng sang các hệ thống động học phức tạp hơn.

Sai lầm thường gặp 2.1: Hạn chế về tính toàn vẹn của phương pháp giải tích thủ công

Một rủi ro phương pháp luận thường gặp là việc tính toán giải tích thủ công đối với các biểu thức phi tuyến nhiều biến số rất dễ phát sinh sai số đại số. Thao tác khai triển đạo hàm riêng thường dẫn đến việc nhầm lẫn về dấu hoặc bỏ sót số hạng. Những khiếm khuyết toán học này, nếu không được kiểm chứng chéo độc lập, sẽ làm sai lệch đáng kể khoảng tin cậy (confidence interval) của kết quả khi mô hình được áp dụng trên tập dữ liệu hàng nghìn điểm lấy mẫu từ các thiết bị đo lường tự động.

Để giảm thiểu tối đa rủi ro từ các thao tác thủ công và tối ưu hóa độ tin cậy của thuật toán, kỹ thuật hệ thống đại số máy tính (computer algebra system) trên môi trường Python được tích hợp nhằm tự động hóa quy trình vi phân. Thư viện sympy cho phép tính toán trực tiếp trên các biểu thức tượng trưng (symbolic expressions), đảm bảo tính toàn vẹn của cấu trúc toán học trước khi chuyển giao tham số vào các phép toán vector hóa (vectorized operations) nhằm xử lý chuỗi mảng dữ liệu.

Python Implementation: Automated Partial Derivative Extraction using SymPy

```
1 import sympy as sp
2 from uncertainties import unumpy as unp
3
4 # Define symbolic variables for the partial derivative extraction
5 s_i, s_j, t_i, t_j = sp.symbols('s_i s_j t_i t_j')
6 v_expr = (s_j - s_i) / (t_j - t_i)
7
8 # Automate partial derivative extraction to verify theoretical error propagation
9 print("Analytical Partial Derivative w.r.t final position (s_j):")
10 sp.pprint(sp.diff(v_expr, s_j))
11 print("\nAnalytical Partial Derivative w.r.t final time (t_j):")
12 sp.pprint(sp.diff(v_expr, t_j))
13 print("\n" + "="*50 + "\n")
```

2.2 Tự động hóa lan truyền sai số với thư viện uncertainties

Trong phân tích dữ liệu thực nghiệm quy mô lớn, việc áp dụng phương pháp giải tích truyền thống cho từng điểm đo để đánh giá mức độ lan truyền độ bất định (error propagation) thường vấp phải giới hạn về khối lượng tính toán và hiệu suất. Nhằm giải quyết rào cản này trong các hệ thống xử lý dữ liệu cảm biến tự động, ta có thể áp dụng thư viện `uncertainties` thuộc hệ sinh thái Python. Đây là một công cụ chuyên biệt được thiết kế để tự động hóa quy trình đánh giá độ bất định thông qua phương pháp vi phân tuyến tính cục bộ.

Cơ sở vật lý 2.1: Mảng dữ liệu mang sai số trong không gian lập trình

Khác biệt với các kiểu mảng vô hướng (scalar arrays) tiêu chuẩn, thư viện `uncertainties` (thông qua mô đun `con unumpy`) biểu diễn các tập dữ liệu đo lường vật lý dưới dạng cấu trúc mảng mang sai số (uncertainty-bearing arrays) thông qua đối tượng `uarray`. Mỗi phần tử trong mảng này tích hợp đồng thời hai trường thông tin: giá trị danh định (nominal value) và độ bất định tuyệt đối (absolute uncertainty) đặc trưng cho giới hạn phân giải của thiết bị. Khi các mảng này tham gia vào những phép toán đại số hoặc vi phân, thuật toán cốt lõi sẽ tự động tính toán ma trận Jacobi và lan truyền sai số tương ứng cho toàn bộ mảng kết quả tại thời điểm thực thi.

Khi áp dụng phương pháp này vào bài toán minh họa động học rơi tự do, mảng dữ liệu tọa độ và thời gian được tích hợp trực tiếp các giới hạn bất định từ thiết bị. Dựa trên thông số kỹ thuật của hệ thống CASSY, giới hạn độ phân giải không gian (spatial resolution) được xác định ở mức 1 mm ($\sigma_s = 0.001$ m), trong khi giới hạn độ phân giải thời gian (temporal resolution) đạt 0.5 ms ($\sigma_t = 0.0005$ s).

Trong môi trường lập trình, việc chuyển đổi từ không gian tọa độ sang mảng vận tốc tức thời được thực hiện thông qua thuật toán vi phân số học. Trong quá trình này, các phép toán vector hóa trên đối tượng `uarray` sẽ tự động định tuyến và tổng hợp phương sai qua từng bước tính toán, ước lượng ra một mảng vận tốc hoàn chỉnh bao gồm độ bất định độc lập cho từng điểm dữ liệu.

Python Implementation: Automated error propagation during numerical differentiation

```

1  # Assign standard instrumental uncertainties based on CASSY sensor limitations
2  sigma_s = 0.001    # Estimated spatial resolution: 1 mm
3  sigma_t = 0.0005   # Estimated temporal resolution: 0.5 ms
4
5  # Initialize uncertainty-bearing arrays
6  s_u = unp.uarray(displacement_data, sigma_s)
7  t_u = unp.uarray(time_data, sigma_t)
8
9  # Automate error propagation during numerical differentiation
10 ds_u = s_u[1:] - s_u[:-1]
11 dt_u = t_u[1:] - t_u[:-1]
12 v_u = ds_u / dt_u

```

```

13
14 # Extract nominal values and dynamically computed uncertainties
15 velocity_vals = unp.nominal_values(v_u)
16 velocity_errs = unp.std_devs(v_u)
17
18 print(f"Propagated Velocity Example [Index 5]: {v_u[5]:.4u} m/s")

```

Việc trích xuất các mảng giá trị danh định thông qua hàm `unp.nominal_values()` và mảng độ bất định thông qua `unp.std_devs()` là thao tác chuẩn bị quan trọng. Sau khi trích xuất, dữ liệu được sắp xếp lại để tương thích với các hàm hồi quy (curve fitting) và các kỹ thuật trực quan hóa ở những bước tiếp theo.

Sai lầm thường gặp 2.2: *Hiện tượng mất đồng bộ chỉ mục (index) giữa các mảng dữ liệu*

Một rủi ro phương pháp luận thường gặp khi xử lý dữ liệu vật lý tự động hóa là việc duy trì song song và độc lập mảng giá trị danh định và mảng độ bất định. Cách xử lý tách riêng hai mảng này dễ gây mất đồng bộ dữ liệu và mất đồng bộ chỉ mục (index desynchronization) khi hệ thống áp dụng các bộ lọc dữ liệu. Phương thức tiếp cận thông qua đối tượng `uarray` giúp giải quyết vấn đề này bằng cách đóng gói hai thực thể toán học vào một phần tử duy nhất, từ đó duy trì tính nhất quán của dữ liệu và sai số qua từng bước xử lý.

Việc chuyển từ tính toán thủ công sang xử lý mảng tự động hóa mang lại lợi ích đáng kể về hiệu suất phân tích. Việc áp dụng trực tiếp các phép vi phân dựa trên đối tượng tích hợp sai số giúp giảm nguy cơ sai sót đại số (algebraic errors), cải thiện tính minh bạch của mã nguồn (code readability) và nâng cao khả năng tái lập (reproducibility) của thực nghiệm, đặc biệt khi quy mô tập dữ liệu được mở rộng.

Hồi quy động học và ma trận hiệp phương sai

3.1 Thuật toán hồi quy và trích xuất ma trận hiệp phương sai

Trong phân tích dữ liệu thực nghiệm tự động hóa, sau khi các mảng dữ liệu thô được nạp từ các tệp lưu trữ ngoại vi (như định dạng .csv), việc ước lượng các hằng số vật lý cốt lõi (như gia tốc trọng trường g) đòi hỏi việc khớp dữ liệu đo với các mô hình toán học. Đối với hệ thống lấy mẫu tự động, dữ liệu đầu vào mang những mức độ bất định hệ thống nhất định. Do đó, quá trình khớp đường cong (curve fitting) không chỉ nhằm mục đích ước lượng quỹ đạo trung tâm, mà còn đóng vai trò ước lượng độ bất định của các tham số thông qua ma trận hiệp phương sai.

Cơ sở vật lý 3.1: Nguyên lý tối ưu hóa bình phương tối thiểu trọng số

Hàm tối ưu hóa hồi quy phi tuyến trong các thư viện phân tích khoa học vận hành dựa trên nguyên lý bình phương tối thiểu trọng số (weighted least squares). Đối với tập dữ liệu động học có chứa phương sai đo lường σ_i^2 , thuật toán tiến hành cực tiểu hóa hàm tổng bình phương phần dư được chuẩn hóa $S(\theta)$:

$$S(\theta) = \sum_{i=1}^N \left[\frac{y_i - f(x_i, \theta)}{\sigma_i} \right]^2 \quad (3.1)$$

Trong đó, $f(x_i, \theta)$ biểu thị mô hình lý thuyết (hàm bậc hai đối với tọa độ hoặc hàm tuyến tính đối với vận tốc). Việc sử dụng trọng số nghịch đảo $1/\sigma_i$ đảm bảo rằng các điểm dữ liệu sở hữu độ bất định cao sẽ có mức độ đóng góp thấp hơn vào quá trình ảnh hưởng đến kết quả hồi quy, từ đó phản ánh đúng mức độ tin cậy của từng điểm đo.

Quá trình giải bài toán cực trị bằng các thuật toán lặp đa chiều (như thuật toán Levenberg-Marquardt) sẽ trả về bộ tham số tối ưu (optimal parameters) và ma trận hiệp phương sai (covariance matrix). Ma trận này, thường được định danh là biến pcov trong môi trường lập trình, đóng vai trò là cơ sở cho việc phân tích sai số cho các phép đo gián tiếp.

Khai triển toán học 3.1: Cấu trúc giải tích của ma trận hiệp phương sai

Ma trận hiệp phương sai Σ (tương ứng đối tượng pcov) cho một mô hình có M tham số tự do là một ma trận vuông đối xứng cấp M . Thông tin về độ bất định và tương quan sai số được thể hiện qua các phần tử:

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \text{cov}_{1,2} & \cdots & \text{cov}_{1,M} \\ \text{cov}_{2,1} & \sigma_2^2 & \cdots & \text{cov}_{2,M} \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}_{M,1} & \text{cov}_{M,2} & \cdots & \sigma_M^2 \end{pmatrix} \quad (3.2)$$

Trong đó, phần tử trên đường chéo chính $\Sigma_{ii} = \sigma_i^2$ là phương sai (variance) của tham số hồi quy thứ i . Độ lệch chuẩn tuyệt đối của tham số (ví dụ: gia tốc trọng trường g) được tính bằng căn bậc hai của phần tử đường chéo tương ứng:

$$u_{\theta_i} = \sqrt{\Sigma_{ii}} \quad (3.3)$$

Các phần tử ngoài đường chéo Σ_{ij} biểu thị mức độ tương quan hiệp phương sai (covariance) giữa các đại lượng.

Thư viện `scipy.optimize` cung cấp hàm `curve_fit` để tự động hóa thuật toán trên. Khi áp dụng vào bài toán minh họa rơi tự do, phương trình $s(t) = 0.5gt^2 + v_0t + s_0$ được viết dưới dạng hàm Python. Trọng số sai số được tích hợp trực tiếp thông qua đối số `sigma`. Các phần tử trên đường chéo ma trận được trích xuất tự động qua các hàm tính toán mảng (vectorized operations) của thư viện `numpy`.

Python Implementation: Kinematic regression and covariance matrix extraction

```
1 from scipy.optimize import curve_fit
2
3 # Define Theoretical Models for Free Fall Kinematics
4 def quadratic_displacement_model(t, g, v0, s0):
5     # s(t) = 0.5 * g * t^2 + v0 * t + s0
6     return 0.5 * g * t**2 + v0 * t + s0
7
8 def linear_velocity_model(t, g, v0):
9     # v(t) = g * t + v0
10    return g * t + v0
11
12 # Execute non-linear curve fitting for Displacement data
13 popt_s, pcov_s = curve_fit(
14     quadratic_displacement_model,
15     time_data,
16     displacement_data,
```



```

17     sigma=np.full_like(displacement_data, sigma_s),
18     absolute_sigma=True,
19     p0=[9.8, 0.0, 0.0]
20 )
21
22 # Extract optimized parameters and their absolute standard deviations
23 g_s_opt, v0_s_opt, s0_opt = popt_s
24 g_s_err = np.sqrt(np.diag(pcov_s))[0]

```

Sai lầm thường gặp 3.1: Sai lệch ma trận hiệp phương sai khi bỏ qua đối số sai số tuyệt đối

Khi cấu hình `curve_fit` cần chú ý rằng sai lệch ước lượng độ bất định nếu bỏ sót tham số `absolute_sigma=True`. Mặc định, hàm này giả định các giá trị trong mảng `sigma` chỉ mang ý nghĩa trọng số tương đối và tiến hành chuẩn hóa ma trận hiệp phương sai dựa trên phương sai của phần dư. Đối với dữ liệu lấy mẫu từ cảm biến, đại lượng độ bất định (như `sigma_s`) phản ánh giới hạn độ phân giải vật lý thực tế của thiết bị. Việc không kích hoạt `absolute_sigma=True` sẽ khiến kết quả sai số không còn phản ánh đúng giới hạn thiết bị, khiến độ lệch chuẩn được trích xuất từ `pcov_s` không còn tương ứng với sai số thực của thiết bị của thiết bị phần cứng.

3.2 Hồi quy tuyến tính và đánh giá chỉ số R-bình phương

Trong hệ thống xử lý dữ liệu cảm biến tự động, phương pháp hồi quy không chỉ được giới hạn ở các phương trình bậc cao đối với tọa độ không gian. Trong bài toán minh họa, vận tốc tức thời $v(t)$ – đại lượng được tính trước đó bằng phương pháp vi phân số từ mảng dữ liệu gốc – tuân theo một mô hình tuyến tính đơn giản. Việc áp dụng thuật toán hồi quy tuyến tính (linear regression) lên tập dữ liệu này cung cấp một cách kiểm tra độc lập cho toàn bộ quy trình xử lý dữ liệu (internal consistency) của toàn bộ quy trình đo lường.

Khai triển toán học 3.2: Mô hình hồi quy tuyến tính cho đại lượng dẫn xuất

Dựa trên phương trình chuyển động cơ bản, vận tốc tức thời v của một vật thể rơi tự do biến thiên theo hàm bậc nhất đối với chuỗi thời gian t :

$$v(t) = gt + v_0 \quad (3.4)$$

Khi đối chiếu phương trình 3.4 với cấu trúc hồi quy tuyến tính tổng quát $y = ax + b$, ta có thể rút ra hệ quả sau:

- **Hệ số góc (Slope) a :** Tương ứng với tham số gia tốc trọng trường g . Đây là một phương pháp độc lập để xác định g , giúp kiểm tra chéo với kết quả từ mô hình hồi quy phi tuyến của tọa độ.
- **Tung độ gốc (Intercept) b :** Tương ứng với tham số vận tốc ban đầu v_0 . Việc ghi nhận bất kỳ

sự sai lệch nào của v_0 so với giá trị 0 (giả định vật được giải phóng từ trạng thái nghỉ) đều cung cấp cơ sở để đánh giá độ trễ kích hoạt của cảm biến (trigger delay).

Tương tự như quy trình tối ưu hóa trước, hàm `curve_fit` tiếp tục được tái sử dụng để hồi quy mô hình với tập dữ liệu vận tốc. Điểm quan trọng của bước này là việc nạp trực tiếp mảng độ bất định của vận tốc (`velocity_errs`) – vốn đã được lan truyền tự động thông qua đối tượng `uarray` – làm trọng số thống kê thông qua đối số `sigma`.

Python Implementation: Linear regression of derived velocity

```

1  # Execute linear curve fitting for Velocity data
2  popt_v, pcov_v = curve_fit(
3      linear_velocity_model,
4      time_midpoints,
5      velocity_vals,
6      sigma=velocity_errs,
7      absolute_sigma=True,
8      p0=[9.8, 0.0]
9  )
10
11 g_v_opt, v0_v_opt = popt_v
12 g_v_err = np.sqrt(np.diag(pcov_v))[0]
13
14 # R2 for Displacement
15 ss_res_s = np.sum((displacement_data - quadratic_displacement_model(time_data,
16     ↪ *popt_s))**2)
17 ss_tot_s = np.sum((displacement_data - np.mean(displacement_data))**2)
18 r2_s = 1 - (ss_res_s / ss_tot_s)
19
20 # R2 for Velocity
21 ss_res_v = np.sum((velocity_vals - linear_velocity_model(time_midpoints,
22     ↪ *popt_v))**2)
23 ss_tot_v = np.sum((velocity_vals - np.mean(velocity_vals))**2)
24 r2_v = 1 - (ss_res_v / ss_tot_v)
25
26 print(f"Gravitational Acceleration (Displacement Fit): {g_s_opt:.3f} +/-
27     ↪ {g_s_err:.3f} m/s2")
28 print(f"Gravitational Acceleration (Velocity Fit): {g_v_opt:.3f} +/- {g_v_err:.3f}
29     ↪ m/s2")

```

Bên cạnh ma trận hiệp phương sai nhằm đánh giá độ bất định của các tham số, hệ số xác định (coefficient of determination - R^2) là một chỉ số dùng để đánh giá mức độ phù hợp của mô hình của đường cong hồi quy. Giá trị R^2 biểu thị tỷ lệ phương sai của biến phụ thuộc có thể được giải thích bởi mô hình lý thuyết. Đối với

đối với một mô hình vật lý có độ phù hợp tốt, giá trị R^2 thường rất gần 1 trong các bài toán động học ít nhiễu.

Sai lầm thường gặp 3.2: Sai lệch hệ thống khi cường chế tung độ gốc của mô hình tuyến tính

Một rủi ro phương pháp luận đáng chú ý là việc chủ quan cấu hình hàm số để loại bỏ tham số tự do (ví dụ: ép mô hình về dạng $v(t) = gt$). Việc ép đường hồi quy đi qua gốc tọa độ mà thiếu cơ sở kiểm định sẽ làm mất đi khả năng phát hiện các sai lệch hệ thống về thời gian đồng bộ (synchronization error) của thiết bị đo. Hậu quả là thuật toán tối ưu hóa sẽ tự động bù trừ sai số này vào hệ số góc, dẫn đến việc ước lượng thông số chính (như gia tốc g) bị sai lệch nghiêm trọng.

3.3 Trực quan hóa mô hình hồi quy động học

Trong phân tích dữ liệu cảm biến tự động, sau khi thuật toán `curve_fit` hoàn tất việc ước lượng các tham số vật lý và tính ma trận hiệp phương sai, việc vẽ đồ thị hồi quy là bước kiểm tra trực quan quan trọng. Thao tác này thể hiện mức độ phù hợp giữa mô hình toán học dự đoán và tập dữ liệu thực nghiệm (bao gồm các điểm dữ liệu và thanh sai số tương ứng). Khi áp dụng vào bài toán minh họa, ta tiến hành vẽ đồ thị song song cho cả mô hình phi tuyến đối với độ dịch chuyển và mô hình tuyến tính đối với vận tốc.

Cơ sở vật lý 3.2: Độ phân giải trực hoành trong nội suy đồ thị

Trong môi trường vẽ đồ thị khoa học của Python, một hàm toán học liên tục không được hiển thị trực tiếp thông qua một công thức giải tích. Thay vào đó, hàm này được xấp xỉ bằng một tập hợp các điểm rời rạc nối tiếp nhau. Nếu mảng thời gian lấy mẫu (`time_data`) được sử dụng trực tiếp làm trục hoành để vẽ đường mô hình hồi quy, đặc biệt đối với các hệ thống có tần số lấy mẫu thưa thớt, đường cong biểu diễn dễ xuất hiện hiện tượng đứt gãy dạng đa giác (polygonal artifacts). Để duy trì tính liên tục trơn tru của hàm phi tuyến về mặt thị giác, việc thiết lập một lưới điểm nội suy mới với độ phân giải cao hơn (thông qua các hàm tạo mảng như `np.linspace`) là một bước cần thiết.

Đoạn mã dưới đây minh họa phương pháp kết hợp đồng thời hai mô hình hồi quy động học trên một cấu trúc đồ thị phân nhánh. Các thông số thống kê cốt lõi, bao gồm phương trình mô hình tối ưu, ước lượng gia tốc trọng trường g kèm theo độ lệch chuẩn và hệ số xác định R^2 , được tự động chèn vào các hộp thông tin (parameter boxes) thông qua chuỗi định dạng (f-strings) kết hợp cú pháp LaTeX.

Python Implementation: Rendering kinematic regression models

```
1 # Visual configuration for Regression Models
2 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
3
4 # Professional square box for parameters
5 param_box_props = dict(boxstyle='square,pad=0.5', facecolor='white',
6                          alpha=0.9, edgecolor='black', linewidth=0.8)
```

```

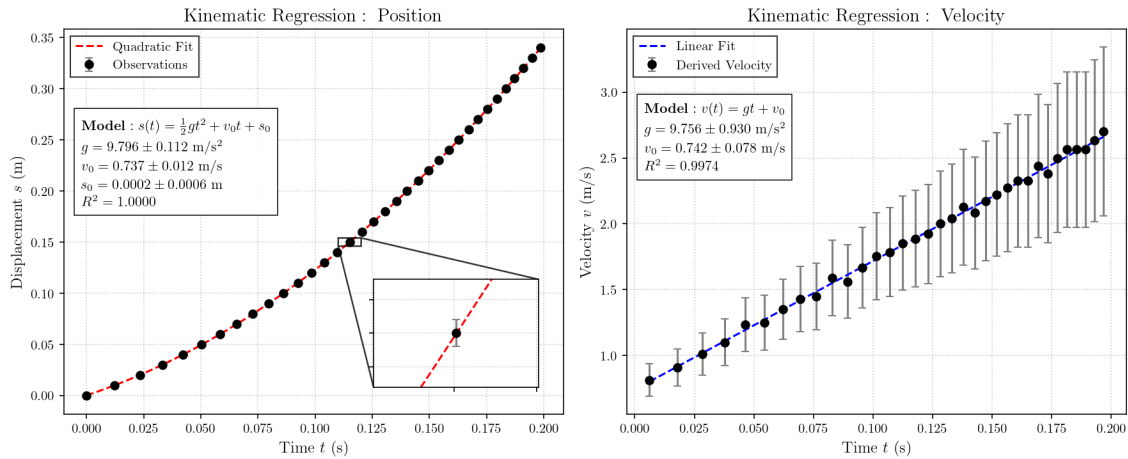
7
8 # Render Displacement Quadratic Regression
9 t_fit_s = np.linspace(min(time_data), max(time_data), 200)
10 ax1.errorbar(time_data, displacement_data, yerr=sigma_s, fmt='o', color='black',
11              ecolord='gray', capsize=3, label=r'$\mathrm{Observations}$')
12 ax1.plot(t_fit_s, quadratic_displacement_model(t_fit_s, *popt_s), 'r--',
13          label=r'$\mathrm{Quadratic \ Fit}$')
14
15 param_text_s = (
16     r'$\mathbf{Model:} \$ $s(t) = \frac{1}{2}gt^2 + v_0t + s_0$' + '\n' +
17     fr'$g = {g_s_opt:.3f} \pm {g_s_err:.3f} \ \mathrm{{m/s^2}}$' + '\n' +
18     fr'$v_0 = {v0_s_opt:.3f} \pm {v0_s_err:.3f} \ \mathrm{{m/s}}$' + '\n' +
19     fr'$R^2 = {r2_s:.4f}$'
20 )
21 ax1.legend(loc='upper left', frameon=True, edgecolor='black', fancybox=False)
22 ax1.text(0.035, 0.77, param_text_s, transform=ax1.transAxes, fontsize=10.5,
23          verticalalignment='top', linespacing=1.5, bbox=param_box_props)
24 ax1.set_xlabel(r'$\mathrm{Time} \ \ t \ \ (\mathrm{s})$')
25 ax1.set_ylabel(r'$\mathrm{Displacement} \ \ s \ \ (\mathrm{m})$')
26 ax1.set_title(r'$\mathrm{Kinematic \ Regression: \ Position}$')
27 ax1.grid(True, linestyle=':', alpha=0.7)
28
29 # Render Velocity Linear Regression
30 t_fit_v = np.linspace(min(time_midpoints), max(time_midpoints), 200)
31 ax2.errorbar(time_midpoints, velocity_vals, yerr=velocity_errs, fmt='o',
32              color='black',
33              ecolord='gray', capsize=3, label=r'$\mathrm{Derived \ Velocity}$')
34 ax2.plot(t_fit_v, linear_velocity_model(t_fit_v, *popt_v), 'b--',
35          label=r'$\mathrm{Linear \ Fit}$')
36
37 param_text_v = (
38     r'$\mathbf{Model:} \$ $v(t) = gt + v_0$' + '\n' +
39     fr'$g = {g_v_opt:.3f} \pm {g_v_err:.3f} \ \mathrm{{m/s^2}}$' + '\n' +
40     fr'$v_0 = {v0_v_opt:.3f} \pm {v0_v_err:.3f} \ \mathrm{{m/s}}$' + '\n' +
41     fr'$R^2 = {r2_v:.4f}$'
42 )
43 ax2.legend(loc='upper left', frameon=True, edgecolor='black', fancybox=False)
44 ax2.text(0.035, 0.81, param_text_v, transform=ax2.transAxes, fontsize=10.5,
45          verticalalignment='top', linespacing=1.5, bbox=param_box_props)
46 ax2.set_xlabel(r'$\mathrm{Time} \ \ t \ \ (\mathrm{s})$')
47 ax2.set_ylabel(r'$\mathrm{Velocity} \ \ v \ \ (\mathrm{m/s})$')
48 ax2.set_title(r'$\mathrm{Kinematic \ Regression: \ Velocity}$')

```

```

48 ax2.grid(True, linestyle=':', alpha=0.7)
49
50 plt.tight_layout()
51 plt.show()

```



Hình 3.1: Trực quan hóa đồ thị hồi quy kết hợp hộp thông tin tham số.

Sai lầm thường gặp 3.3: Hiện tượng phân kỳ trong thuật toán tối ưu hóa phi tuyến

Khi khởi chạy hàm `curve_fit` cho các mô hình phi tuyến, thuật toán đôi khi trả về ngoại lệ `Optimal parameters not found`. Lỗi này xuất phát từ bản chất của các phương pháp tối ưu hóa lặp (ví dụ: thuật toán Levenberg-Marquardt), vốn rất nhạy cảm với miền tham số. Nếu điểm xuất phát (initial guess) mặc định (được thiết lập qua đối số `p0`) nằm quá xa so với cực tiểu toàn cục, thuật toán dễ bị kẹt tại các cực tiểu cục bộ (local minima) hoặc không hội tụ. Để giảm đáng kể rủi ro này, chương trình cần truyền vào một tập giá trị ban đầu hợp lý về mặt vật lý (ví dụ: `p0=[9.8, 0.0, 0.0]`). Giá trị 9.8 cung cấp cho thuật toán một giá trị khởi tạo hợp lý (lực hấp dẫn), giúp thuật toán hội tụ ổn định hơn và tối ưu hóa thời gian xử lý.

Kỹ thuật chẩn đoán mô hình nâng cao

4.1 Đánh giá mô hình với tiêu chuẩn Chi bình phương rút gọn

Đánh giá mức độ phù hợp của mô hình (goodness-of-fit) là bước cốt lõi trong phân tích dữ liệu thực nghiệm sau khi các tham số động học đã được ước lượng. Trong phương pháp hồi quy áp dụng cho hệ thống đo lường thực nghiệm, việc chỉ đánh giá chất lượng mô hình dựa trên hệ số xác định (coefficient of determination - R^2) có thể chưa phản ánh đầy đủ chất lượng mô hình, do bản thân chỉ số R^2 không phản ánh được giới hạn sai số hệ thống của các thiết bị cảm biến.

Cơ sở vật lý 4.1: Giới hạn của hệ số R^2 và vai trò của Chi bình phương rút gọn

Hệ số R^2 biểu diễn tỷ lệ phương sai của đại lượng phụ thuộc được giải thích bởi mô hình hồi quy. Tuy nhiên, đại lượng này có một số hạn chế trong đo lường vật lý: nó không tích hợp độ bất định (uncertainty) do các ranh giới độ phân giải của thiết bị (instrumental limits) gây ra. Giá trị Chi bình phương rút gọn (reduced chi-square) được áp dụng nhằm giải quyết giới hạn này, giúp đánh giá độ tương thích của đường cong hồi quy trên cơ sở xét đến trọng số bất định của từng điểm dữ liệu cụ thể.

Việc đánh giá độ lệch giữa mô hình và dữ liệu đo thông qua phương sai giúp phát hiện các vấn đề trong mô hình hồi quy.

Khai triển toán học 4.1: Công thức và ý nghĩa thống kê của Chi bình phương rút gọn

Hàm mục tiêu Chi bình phương (χ^2) là tổng bình phương các phần dư (residuals) được chuẩn hóa (normalized) bởi độ bất định đo lường của từng điểm dữ liệu:

$$\chi^2 = \sum_{i=1}^N \left[\frac{s_i - f(t_i, \theta)}{\sigma_s} \right]^2 \quad (4.1)$$

Trong đó, N là tổng số lượng điểm lấy mẫu, s_i là tọa độ thực nghiệm, $f(t_i, \theta)$ là giá trị ước lượng từ hàm hồi quy, và σ_s là độ lệch chuẩn tuyệt đối phản ánh giới hạn đo lường.

Tiêu chuẩn Chi bình phương rút gọn χ^2_ν được thiết lập bằng cách chia giá trị hàm mục tiêu cho số bậc tự do (degrees of freedom) ν :

$$\chi^2_\nu = \frac{\chi^2}{\nu} = \frac{\chi^2}{N - M} \quad (4.2)$$

Với M là số lượng tham số tự do trong hàm hồi quy. Giá trị χ^2_ν cung cấp cơ sở thống kê để đánh giá chất lượng của mô hình:

- $\chi^2_\nu \approx 1$: Mô hình phù hợp tốt với dữ liệu. Độ biến thiên của phần dư phù hợp với mức độ nhiễu ngẫu nhiên kỳ vọng từ giới hạn phân giải thiết bị.
- $\chi^2_\nu \gg 1$: Mô hình chưa mô tả tốt dữ liệu (underfitting) hoặc cho thấy mô hình vật lý có thể chưa đầy đủ.
- $\chi^2_\nu \ll 1$: Mô hình bị overfit (overfitting), hoặc độ bất định σ_s đã bị ước lượng cao hơn thực tế (overestimated) quá mức.

Đoạn mã sau thực hiện việc tính toán thông số chẩn đoán này đối với hàm hồi quy không gian (độ dịch chuyển). Việc thiết lập thuật toán không đòi hỏi các gói thư viện phức tạp mà được cấu trúc hoàn toàn dựa trên các phép toán vector hóa của mô-đun numpy.

Python Implementation: Implement reduced Chi-Square metric for displacement

```
1 # 1. Implement Reduced Chi-Square Metric for the Displacement Model
2 residuals_s = displacement_data - quadratic_displacement_model(time_data, *popt_s)
3 normalized_residuals_s = residuals_s / sigma_s
4 chi_square_s = np.sum(normalized_residuals_s**2)
5 degrees_of_freedom_s = len(displacement_data) - len(popt_s)
6 reduced_chi_square_s = chi_square_s / degrees_of_freedom_s
7
8 print(f"Reduced Chi-Square (Displacement Model): {reduced_chi_square_s:.4f}")
```

Sai lầm thường gặp 4.1: Hiện tượng quá khớp (overfitting) do gia tăng bậc đa thức mô hình

Trong quá trình hiệu chỉnh mô hình, việc tăng bậc của đa thức hồi quy thường được lạm dụng nhằm tối đa hóa chỉ số R^2 . Dù một hàm toán học bậc cao có thể làm đường cong bám sát mọi điểm trong tập dữ liệu hữu hạn, thao tác này tất yếu làm gia tăng số lượng tham số M , làm giảm số bậc tự do ν . Hệ quả là χ^2_ν giảm xuống thấp hơn nhiều so với 1, phản ánh một trạng thái quá khớp (overfitting). Việc cố ý phức tạp hóa cấu trúc toán học để bù đắp nhiễu ngẫu nhiên làm mất ý nghĩa vật lý của mô hình, giảm khả năng khái quát hóa của mô hình.

4.2 Phân tích phần dư và kỹ thuật làm trơn LOWESS

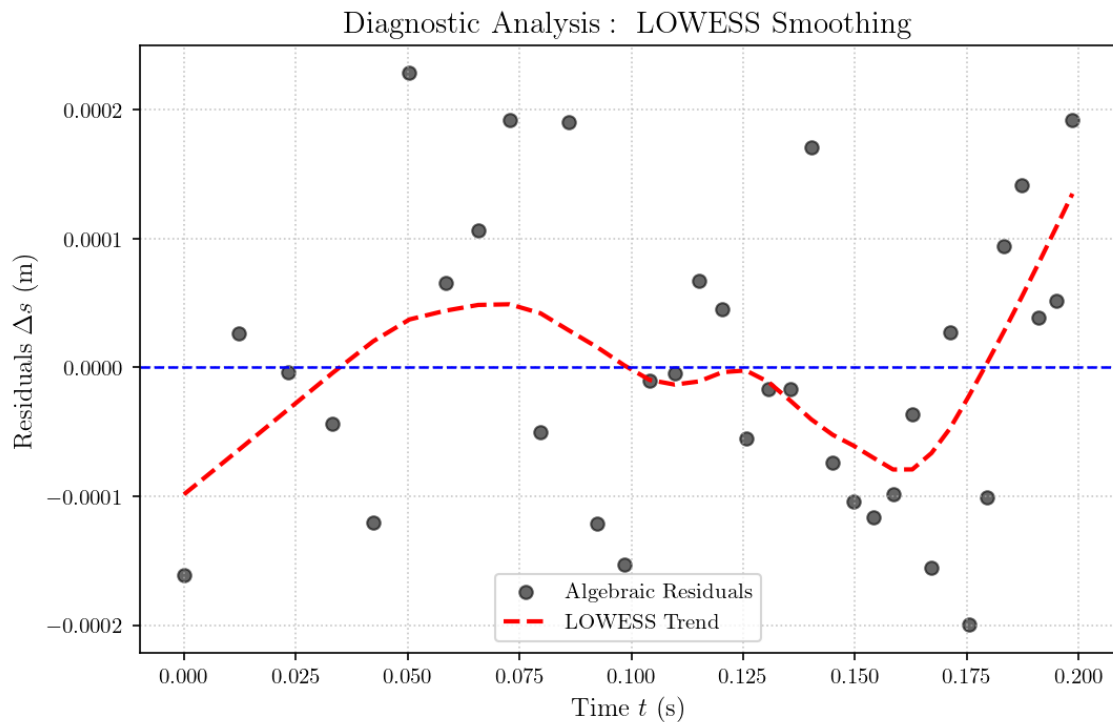
Phân tích phần dư (residual analysis) là phương pháp dùng để phát hiện sai lệch hệ thống chưa được mô hình mô tả. Phần dư (residuals) là chênh lệch đại số giữa giá trị quan trắc thực tế từ thiết bị cảm biến và giá trị tiên đoán từ phương trình lý thuyết. Đối với một hệ thống đo lường lý tưởng, ví dụ như bài toán rơi tự do nơi lực tác dụng duy nhất là trọng lực, phần dư của mô hình tọa độ được kỳ vọng sẽ phân bố ngẫu nhiên quanh trục hoành. Nếu phần dư xuất hiện xu hướng rõ ràng (systematic trend) trên đồ thị phân tán, điều đó phản ánh sự hiện diện của các hiệu ứng vật lý khác tác động lên đối tượng đo lường.

Thuật toán làm trơn LOWESS (Locally Weighted Scatterplot Smoothing) từ thư viện statsmodels được áp dụng để trực quan hóa các xu hướng phi tuyến này. Là một phương pháp hồi quy cục bộ phi tham số (non-parametric local regression), LOWESS tạo ra một đường cong liên tục biểu diễn xu hướng chính của phần dư mà không yêu cầu xác định trước bất kỳ cấu trúc hàm giải tích cụ thể nào.

Python Implementation: Residual analysis and non-parametric LOWESS smoothing

```
1 import statsmodels.api as sm
2
3 # 2. Residual Analysis and Non-parametric LOWESS Smoothing
4 lowess_model = sm.nonparametric.lowess
5 smoothed_residuals = lowess_model(residuals_s, time_data, frac=0.4)
6
7 fig, ax = plt.subplots(figsize=(8, 5))
8 ax.scatter(time_data, residuals_s, color='black', edgecolor='black', alpha=0.6,
9            label=r'$\mathrm{Algebraic \ Residuals}$')
9 ax.plot(smoothed_residuals[:, 0], smoothed_residuals[:, 1], 'r--', linewidth=2,
10         label=r'$\mathrm{LOWESS \ Trend}$')
11 ax.axhline(0, color='blue', linestyle='--', linewidth=1.2)
12 ax.set_xlabel(r'$\mathrm{Time} \ t \ (\mathrm{s})$')
13 ax.set_ylabel(r'$\mathrm{Residuals} \ \Delta s \ (\mathrm{m})$')
14 ax.set_title(r'$\mathrm{Diagnostic \ Analysis: \ LOWESS \ Smoothing}$')
15 ax.legend()
16
17 plt.grid(True, linestyle=':', alpha=0.7)
```

Khi áp dụng thuật toán làm trơn này lên chuỗi phần dư của dữ liệu tọa độ, đường cong LOWESS cung cấp cơ sở thống kê để đánh giá các điều kiện thực nghiệm phi lý tưởng.



Hình 4.1: Phân tích phần dư và chẩn đoán xu hướng hệ thống thông qua kỹ thuật làm trơn LOWESS.

Cơ sở vật lý 4.2: Đánh giá lực cản khí động học qua cấu trúc phần dư

Trong bài toán rơi tự do thực tế, nếu vật thể rơi trong môi trường không khí và có lực cản không khí đáng kể, phương trình lý thuyết sẽ không còn mô tả tốt dữ liệu. Lực cản không khí thường gia tăng theo bình phương vận tốc. Khi vật thể gia tốc theo thời gian, quãng đường thực tế có xu hướng chậm dần so với dự đoán từ hàm bậc hai. Sự sai lệch này làm phát sinh các giá trị phần dư âm ở giai đoạn cuối của quá trình thu thập dữ liệu. Việc đường LOWESS bẻ cong xuống dưới dải không (zero-line) tại các mốc thời gian lớn cho thấy để đánh giá mức độ tác động của lực cản khí động học (aerodynamic drag) đối với mô hình.

Sai lầm thường gặp 4.2: Ảnh hưởng của tham số phân số băng thông ($frac$) đến độ trơn LOWESS

Một vấn đề thường gặp khi sử dụng thuật toán LOWESS là việc cấu hình thiếu hợp lý tham số phân số băng thông (bandwidth fraction), được điều chỉnh thông qua đối số $frac$. Nếu $frac$ quá nhỏ, mô hình sẽ gặp hiện tượng quá khớp (overfitting), khiến đường cong bị gãy khúc do bám sát các dao động ngẫu nhiên sinh ra từ giới hạn phân giải của thiết bị đo. Ngược lại, một giá trị $frac$ tiệm cận 1.0 sẽ gây ra hiện tượng phẳng hóa quá mức (oversmoothing), làm mờ đi đáng kể xu hướng tổng thể phản ánh các yếu tố vật lý (như lực cản không khí). Đối với các tập dữ liệu động học tự động hóa, việc thiết lập mức băng thông khoảng $frac = 0.4$ thường cho kết quả ổn định, giúp đường cong LOWESS thể hiện rõ xu hướng chính mà không bị nhiễu cục bộ chi phối.

4.3 Kiểm chứng độ bất định bằng phương pháp mô phỏng Monte Carlo

Trong phân tích đo lường học thực nghiệm, ma trận hiệp phương sai (covariance matrix) thu được từ thuật toán hồi quy cung cấp một xấp xỉ tuyến tính quanh nghiệm tối ưu (local linearized estimation) đối với độ bất định của các tham số vật lý. Tuy nhiên, khi đối chiếu với các hệ thống có đặc tính phi tuyến cao hoặc khi nhiều đầu vào không còn tuân theo mô hình đơn giản, phương pháp giải tích này có thể bộc lộ những giới hạn nhất định. Do đó, phương pháp mô phỏng Monte Carlo (Monte Carlo simulation) được ứng dụng như một công cụ mô phỏng hiệu quả nhằm kiểm tra chéo (cross-validate) mức độ lan truyền sai số thực tế trong các hệ thống xử lý dữ liệu tự động hóa.

Cơ sở vật lý 4.3: Cơ sở thống kê của phương pháp mô phỏng Monte Carlo

Phương pháp Monte Carlo trong đo lường học dựa trên việc thêm nhiễu ngẫu nhiên vào dữ liệu (random perturbation). Đối với các hệ thống cảm biến lấy mẫu tự động, độ bất định của biến số đầu vào (như độ phân giải thời gian và không gian) được đặc trưng bởi các phân bố chuẩn (Gaussian distributions) quanh giá trị danh định. Trong quy trình này, hàng nghìn tập dữ liệu giả lập (simulated datasets) được tạo bằng cách thêm nhiễu ngẫu nhiên vào mảng dữ liệu thực nghiệm đã được nạp từ tệp .csv. Toàn bộ tập dữ liệu giả lập sau đó được đưa vào thuật toán hồi quy để tạo ra một phân bố thống kê (statistical distribution) đối với tham số vật lý cần ước lượng (trong bài toán minh họa là gia tốc trọng trường g). Các đại lượng thống kê (cụ thể là độ lệch chuẩn) được rút ra từ phân bố này sẽ được dùng làm độ bất định tổng thể của toàn bộ phép đo.

Quá trình giả lập dữ liệu tại từng vòng lặp được mô tả bằng phép cộng tuyến tính giữa tín hiệu thực tế và thành phần nhiễu.

Khai triển toán học 4.2: Mô hình toán học của nhiễu ngẫu nhiên

Tại vòng lặp thứ k , tập dữ liệu tọa độ s_{sim} và thời gian t_{sim} được tái tạo bằng cách cộng thêm nhiễu phân bố chuẩn vào tập dữ liệu gốc:

$$s_{\text{sim},i} = s_i + \mathcal{N}(0, \sigma_s^2) \quad (4.3)$$

$$t_{\text{sim},i} = t_i + \mathcal{N}(0, \sigma_t^2) \quad (4.4)$$

Trong đó, $\mathcal{N}(0, \sigma^2)$ là hàm sinh số giả ngẫu nhiên tuân theo phân bố chuẩn có giá trị trung bình bằng 0 và phương sai tương ứng với bình phương giới hạn độ phân giải của thiết bị đo. Đối với mỗi cặp mảng dữ liệu giả lập thu được, hàm mục tiêu bình phương tối thiểu được tối ưu hóa lại để xác định một giá trị ước lượng $g_{\text{mc}}^{(k)}$.

Đoạn mã dưới đây tự động hóa quy trình phân tích Monte Carlo đối với hằng số gia tốc rơi tự do.

Python Implementation: Monte Carlo simulation for parameter uncertainty

```

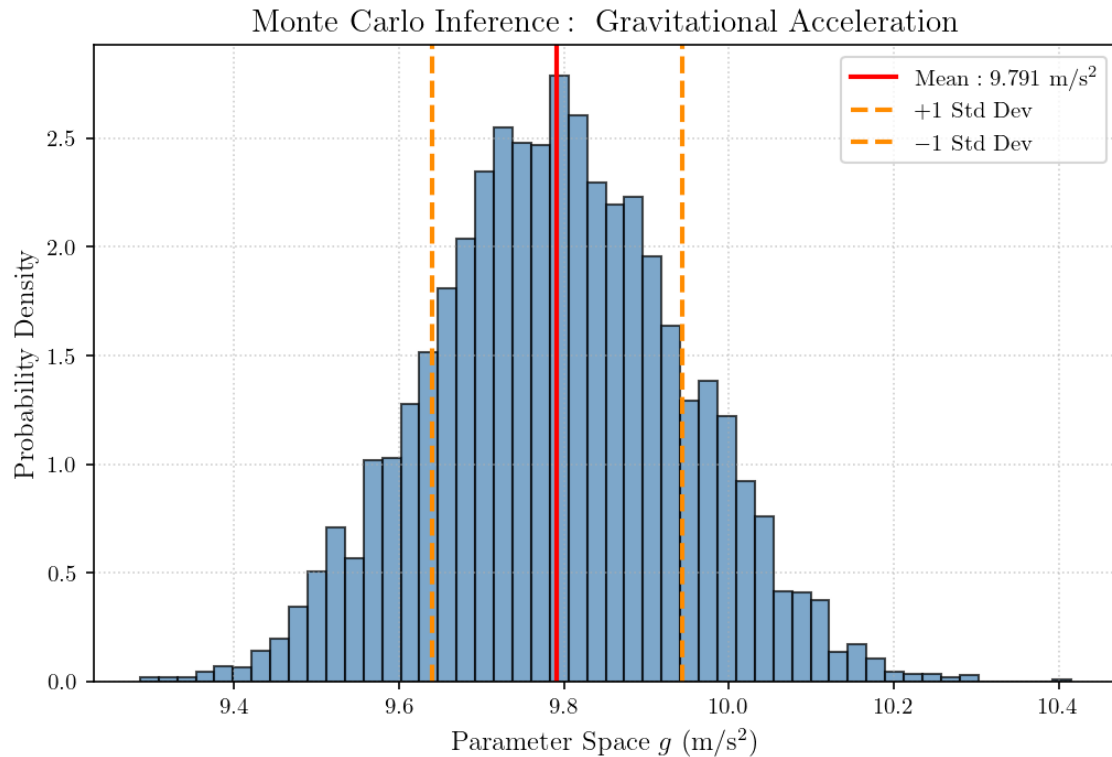
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import curve_fit
4
5 # 3. Monte Carlo Simulation for Parameter Uncertainty (Gravitational Acceleration)
6 num_simulations = 5000
7 np.random.seed(42)
8
9 g_mc_results = np.zeros(num_simulations)
10
11 for i in range(num_simulations):
12     # Inject Gaussian noise strictly aligned with instrumental uncertainties
13     s_simulated = displacement_data + np.random.normal(0, sigma_s,
14     ↪ len(displacement_data))
15     t_simulated = time_data + np.random.normal(0, sigma_t, len(time_data))
16
17     try:
18         popt_mc, _ = curve_fit(
19             quadratic_displacement_model,
20             t_simulated,
21             s_simulated,
22             p0=[9.8, 0.0, 0.0],
23             maxfev=2000
24         )
25         g_mc_results[i] = popt_mc[0]
26     except RuntimeError:
27         g_mc_results[i] = np.nan
28
29 # Compute statistical limits from valid inferences
30 g_mc_clean = g_mc_results[~np.isnan(g_mc_results)]
31 g_mean = np.mean(g_mc_clean)
32 g_std = np.std(g_mc_clean)
33
34 # Histogram Visualization of State Space
35 fig_mc, ax_mc = plt.subplots(figsize=(8, 5))
36 ax_mc.hist(g_mc_clean, bins=50, density=True, color='steelblue', edgecolor='black',
37 ↪ alpha=0.7)
38 ax_mc.axvline(g_mean, color='red', linestyle='-', linewidth=2,
39 ↪ label=fr'$\mathrm{{Mean}}$: {g_mean:.3f} \ \mathrm{{m/s^2}}$')
40 ax_mc.axvline(g_mean + g_std, color='darkorange', linestyle='--', linewidth=2,
41 ↪ label=fr'$+1 \ \mathrm{{Std \ Dev}}$')

```

```

38 ax_mc.axvline(g_mean - g_std, color='darkorange', linestyle='--', linewidth=2,
    ↪ label=r'$-1 \ \mathrm{Std} \ \mathrm{Dev}$')
39
40 ax_mc.set_title(r'$\mathrm{Monte} \ \mathrm{Carlo} \ \mathrm{Inference}: \ \mathrm{Gravitational} \ \mathrm{Acceleration}$')
    ↪ Acceleration}$')
41 ax_mc.set_xlabel(r'$\mathrm{Parameter} \ \mathrm{Space} \ g \ \ (\mathrm{m/s^2})$')
42 ax_mc.set_ylabel(r'$\mathrm{Probability} \ \mathrm{Density}$')
43 ax_mc.legend()
44 ax_mc.grid(True, linestyle=':', alpha=0.6)
45 plt.show()
46
47 print(f"Monte Carlo Inference Result: g = {g_mean:.4f} +/- {g_std:.4f} m/s^2")

```



Hình 4.2: Phân bố của giá trị gia tốc trọng trường thu được thông qua mô phỏng Monte Carlo.

Biểu đồ tần suất (histogram) thu được cho thấy xu hướng tiến gần phân bố chuẩn của tham số gia tốc. Độ lệch chuẩn trích xuất từ mảng giá trị hợp lệ (`g_mc_clean`) được dùng để ước lượng độ bất định tổng thể cho kết quả hồi quy.

Sai lầm thường gặp 4.3: Sự phân kỳ thuật toán và nhiễu thống kê do cỡ mẫu nhỏ

Độ ổn định của kết quả ước lượng từ mô phỏng Monte Carlo phụ thuộc chặt chẽ vào kích thước mẫu (`num_simulations`). Số lượng vòng lặp quá nhỏ dễ gây ra các dao động thống kê, làm ảnh hưởng đến độ tin cậy của độ lệch chuẩn được tính toán. Hơn nữa, việc thêm nhiễu ngẫu nhiên trực tiếp vào mảng dữ liệu có khả năng sinh ra các giá trị không phù hợp về mặt vật lý (ví dụ: vật thể di chuyển lùi trong một bước thời gian cực ngắn do sai số giả lập lớn). Những điểm dữ liệu dị biệt này có thể khiến thuật toán lặp (như Levenberg-Marquardt) từ chối hội tụ, dẫn đến việc phát sinh ngoại lệ `RuntimeError`. Để quá trình mô phỏng ổn định hơn, ta cần đặt hàm `curve_fit` trong khối lệnh `try...except` nhằm xử lý lỗi phân kỳ và gán giá trị rỗng (`np.nan`) cho kết quả thất bại. Cách xử lý này cho phép thuật toán dễ dàng thanh lọc dữ liệu dị biệt (thông qua hàm `~np.isnan`) trước khi trích xuất thống kê cuối cùng. Ngoài ra, việc thiết lập hạt giống khởi tạo bằng hàm `np.random.seed` là yêu cầu thiết yếu để đảm bảo khả năng tái lập (reproducibility) của phép tính.

Sự tương đồng giữa độ bất định thu được từ mô phỏng Monte Carlo (`g_std`) và độ bất định trích xuất trực tiếp từ cấu trúc ma trận hiệp phương sai (`g_s_err`) cho thấy mô hình hồi quy hoạt động ổn định. Mỗi tương quan này cho thấy các giới hạn phân giải của thiết bị đo đã được phản ánh nhất quán trong toàn bộ quá trình tính toán của bài toán động học.

Cẩm nang tra cứu nhanh thuật toán (Methodological Quick Reference)

Phụ lục này tổng hợp các thuật toán cốt lõi được ứng dụng xuyên suốt tài liệu này nhằm tối ưu hóa thời gian triển khai quy trình phân tích dữ liệu đo lường tự động hóa. Các khối mã lệnh được đóng gói dưới dạng mẫu mã tham khảo (reference templates), cho phép tái sử dụng linh hoạt trong mọi quy trình xử lý dữ liệu cảm biến tổng quát. Các bước thực thi trải dài từ khâu nạp dữ liệu ngoại vi (tệp .csv), lan truyền độ bất định, tối ưu hóa hồi quy đến chẩn đoán thống kê nâng cao, trong đó bài toán động học rơi tự do được trình bày như một trường hợp minh họa thực tiễn.

Cơ sở vật lý A.1: Định hướng mô đun hóa trong mã nguồn khoa học

Việc xây dựng hệ thống phân tích dữ liệu thực nghiệm đòi hỏi sự ứng dụng thống nhất nguyên lý mô đun hóa (modularity). Thay vì thiết lập các mã nguồn nguyên khối (monolithic scripts), cấu trúc thuật toán cần được phân tách thành các khối logic độc lập (bao gồm: tiền xử lý, lan truyền sai số, hồi quy, chẩn đoán). Kiến trúc này hỗ trợ tính khả năng mở rộng (portability) khi mở rộng quy trình phân tích từ mô hình minh họa sang các hệ thống vật lý phức tạp hơn, đồng thời giảm thiểu rủi ro phát sinh sai lệch hệ thống trong chuỗi xử lý tự động.

A.1. Tự động hóa lan truyền độ bất định (Automated Error Propagation)

Quy trình chuẩn hóa khởi đầu bằng việc nạp trực tiếp tập dữ liệu từ tệp .csv ngoại vi, tiếp nối bằng thuật toán tính toán đại lượng dẫn xuất (như vận tốc tức thời từ vị trí phân tọa độ). Kỹ thuật này đi kèm với việc bảo toàn ma trận sai số thống kê thông qua cấu trúc uarray của thư viện uncertainties.

Python Implementation: Data ingestion

```
1 import pandas as pd
2 import numpy as np
```

```

3 import sympy as sp
4 from uncertainties import unumpy as unp
5
6 # Standard data ingestion from an external CSV file
7 df = pd.read_csv('kinematics_data.csv')
8 time_data = df['Time_s'].values
9 displacement_data = df['Displacement_m'].values
10
11 # Assign standard instrumental uncertainties based on sensor limitations
12 sigma_s = 0.001 # Estimated spatial resolution: 1 mm
13 sigma_t = 0.0005 # Estimated temporal resolution: 0.5 ms
14
15 # Initialize uncertainty-bearing arrays
16 s_u = unp.uarray(displacement_data, sigma_s)
17 t_u = unp.uarray(time_data, sigma_t)
18
19 # Automate error propagation during numerical differentiation
20 ds_u = s_u[1:] - s_u[:-1]
21 dt_u = t_u[1:] - t_u[:-1]
22 v_u = ds_u / dt_u
23
24 # Extract nominal values and dynamically computed uncertainties
25 velocity_vals = unp.nominal_values(v_u)
26 velocity_errs = unp.std_devs(v_u)

```

A.2. Khớp mô hình và trích xuất ma trận hiệp phương sai (Curve Fitting and Covariance)

Kịch bản nền tảng cho phương pháp tối ưu hóa phi tuyến và tuyến tính, bao gồm việc thiết lập trọng số đo lường học thông qua đối số `absolute_sigma=True` và đánh giá độ bất định của các tham số vật lý trực tiếp từ đường chéo của ma trận hiệp phương sai.

Python Implementation: Kinematic regression templates

```

1 from scipy.optimize import curve_fit
2
3 # Define Theoretical Models for Free Fall Kinematics
4 def quadratic_displacement_model(t, g, v0, s0):
5     #  $s(t) = 0.5 * g * t^2 + v0 * t + s0$ 
6     return 0.5 * g * t**2 + v0 * t + s0

```



```

7
8 def linear_velocity_model(t, g, v0):
9     #  $v(t) = g * t + v0$ 
10    return g * t + v0
11
12 # Execute non-linear curve fitting for Displacement data
13 popt_s, pcov_s = curve_fit(
14     quadratic_displacement_model,
15     time_data,
16     displacement_data,
17     sigma=np.full_like(displacement_data, sigma_s),
18     absolute_sigma=True,
19     p0=[9.8, 0.0, 0.0]
20 )
21
22 # Extract optimized parameters and their absolute standard deviations
23 g_s_opt, v0_s_opt, s0_opt = popt_s
24 g_s_err = np.sqrt(np.diag(pcov_s))[0]

```

A.3. Chẩn đoán thống kê và phân tích tập giá trị mô phỏng (Statistical Diagnostics)

Hệ thống mã lệnh chẩn đoán độc lập nhằm kiểm định mức độ tương thích của mô hình thông qua tham số *Reduced Chi-Square*, nhận diện xu hướng của phần dư thông qua thuật toán làm trơn phi tham số *LOWESS*, và đánh giá mức độ hội tụ của các tham số vật lý (như gia tốc g) bằng kỹ thuật thiết lập tập giá trị mô phỏng *Monte Carlo*.

Python Implementation: Advanced statistical evaluation and Monte Carlo inference

```

1 import statsmodels.api as sm
2
3 # 1. Implement Reduced Chi-Square Metric for the Displacement Model
4 residuals_s = displacement_data - quadratic_displacement_model(time_data, *popt_s)
5 normalized_residuals_s = residuals_s / sigma_s
6 chi_square_s = np.sum(normalized_residuals_s**2)
7 degrees_of_freedom_s = len(displacement_data) - len(popt_s)
8 reduced_chi_square_s = chi_square_s / degrees_of_freedom_s
9
10 # 2. Residual Analysis and Non-parametric LOWESS Smoothing

```

```

11 lowess_model = sm.nonparametric.lowess
12 smoothed_residuals = lowess_model(residuals_s, time_data, frac=0.4)
13
14 # 3. Monte Carlo Simulation for Parameter Uncertainty (Gravitational Acceleration)
15 num_simulations = 5000
16 np.random.seed(42)
17
18 g_mc_results = np.zeros(num_simulations)
19
20 for i in range(num_simulations):
21     # Inject Gaussian noise strictly aligned with instrumental uncertainties
22     s_simulated = displacement_data + np.random.normal(0, sigma_s,
23     ↪ len(displacement_data))
24     t_simulated = time_data + np.random.normal(0, sigma_t, len(time_data))
25
26     try:
27         popt_mc, _ = curve_fit(
28             quadratic_displacement_model,
29             t_simulated,
30             s_simulated,
31             p0=[9.8, 0.0, 0.0],
32             maxfev=2000
33         )
34         g_mc_results[i] = popt_mc[0]
35     except RuntimeError:
36         g_mc_results[i] = np.nan
37
38 # Compute statistical limits from valid inferences
39 g_mc_clean = g_mc_results[~np.isnan(g_mc_results)]
40 g_mean = np.mean(g_mc_clean)
41 g_std = np.std(g_mc_clean)

```

Sai lầm thường gặp A.1: Xung đột không gian tên khi ghép nối mã nguồn (Namespace Collision)

Khi thao tác lắp ráp các đoạn mã nguyên mẫu từ cẩm nang tra cứu vào một kịch bản phân tích quy mô lớn, rủi ro ghi đè biến (variable overwriting) dễ xảy ra nếu không gian tên (namespace) không được quản lý chặt chẽ. Việc sử dụng lặp lại các định danh (như mảng `time_data` hoặc bộ tham số `popt_s`) qua nhiều lượt thử nghiệm trên các tập dữ liệu ngoại vi độc lập có thể dẫn đến hiện tượng rò rỉ dữ liệu (data leakage). Hệ quả là các thông số vật lý được ước lượng có khả năng mang giá trị sai lệch đáng kể so với bản chất thực nghiệm của hệ thống đo.

Tài liệu tham khảo